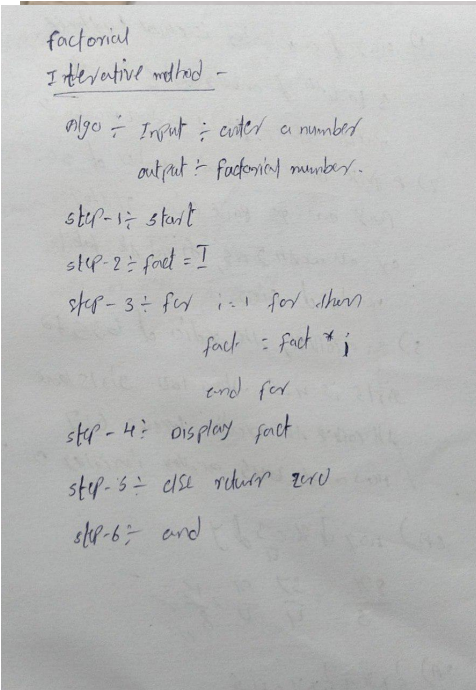


PRACTICAL-04

AIM: Implementation and Time analysis of factorial program using iterative and recursive method

1.) ITERATIVE METHOD

ALGORITHM:



factorial
Iterative method -
algo :- Input :- enter a number
 output :- factorial number.
step-1 :- start
step-2 :- fact = 1
step-3 :- for i = 1 for return
 fact = fact * i
 end for
step-4 :- display fact
step-5 :- else return zero
step-6 :- end

PROGRAM:

```
#include <iostream>

using namespace std;

int main(){

    int n;

    int fact = 1;

    cout << "enter a number:";

    cin >> n;

    for(int i = 1; i <= n; i++){

        fact = fact * i;

    }

    cout << "Factorial of " << n << " = " << fact;
```

```
return 0;
```

```
}
```

OUTPUT:

```
Output
enter a number:5
Factorial of 5 = 120

=== Code Execution Successful ===
```

TIME COMPLEXITY:

Time Complexity:

fact = 1

for (i = 1 to n)

fact = fact * i

$T(n) = 1 + n + n = 2n + 1$

$T(n) = O(n)$

Average case

loop always executes n times

i = 1 → 1

i = 2 → 1

i = 3 → 1

i = 4 → 1

no. of iterations: n

$T(n) = O(n)$

Best cases

loop runs from 1 to n

fact = 1 → 1

for i = 1 to n → n

fact = fact * i → n

$T(n) = 1 + n + n$

⇒ $T(n) = O(n)$

Worst case

It is also, loop's executes n times

n → n/2 ... n/4 ...

1, 2, 3, ... n

$T(n) = 1 + n + n = 2n + 1$

$T(n) = O(n)$

2.) RECURSIVE METHOD:

ALGORITHM:

Recursive method :-

algo :-

Input :- enter a number

output :- factorial number.

step-1 :- start

step-2 :- factorial (n)

step-3 :- fact = 1 or n = 1 then return 1

step-4 :- else fact = n * fact(n-1)

step-5 :- return fact

step-6 :- call factorial (n)

step-7 :- end

PROGRAM:

```
#include <iostream>
```

```
using namespace std;
```

```
class factorial{
```

```
public:
```

```
int factr(int n){
```

```
if(n == 0 || n == 1){
```

```
    return 1;
```

```
}
```

```
else{
```

```
    int result = n*factr(n-1);
```

```
    return result;
```

```
}
```

```
}
```

```
};

int main(){

    int n,result;

    factorial obj;

    cout << "enter a number:";

    cin>>n;

    result = obj.factr(n);

    cout << "factorial of " << n << " is " << result;

    return 0;

}
```

OUTPUT:

```
Output

enter a number:7
factorial of 7 is 5040

=== Code Execution Successful ===
```

TIME COMPLEXITY:

Time Complexity :

factorial (n) $\rightarrow n$
if (n = 0 or n = 1) $\rightarrow n$
 $n \times \text{factorial}(n-1) \rightarrow (n-1)$
return fact

$T(n) = n + n + (n-1) + n$
 $= 4n+1$
 $T(n) = O(n)$

Best case :
when $n=0$ or $n=1$
factorial (1) $\rightarrow 1$
 $T(n) = O(1)$

Average case :
No. of calls is 5
ex: $n=5 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$
 $T(n) = O(n)$

Worst case :
reverse number of n continues until it reaches 1
ex: $n \rightarrow n \rightarrow 1 \rightarrow n-2 \dots 1$ $T(n) = O(n)$

CONCLUSION:

Both iterative and recursive methods are used to find factorial. Both take $O(n)$ time, but the iterative method uses less memory and is simpler to implement.